ASIAN INSTITUTE OF MANAGEMENT

Machine Learning 3

Convolutional Neural Networks

Sessions 7-8

MSDS 2026

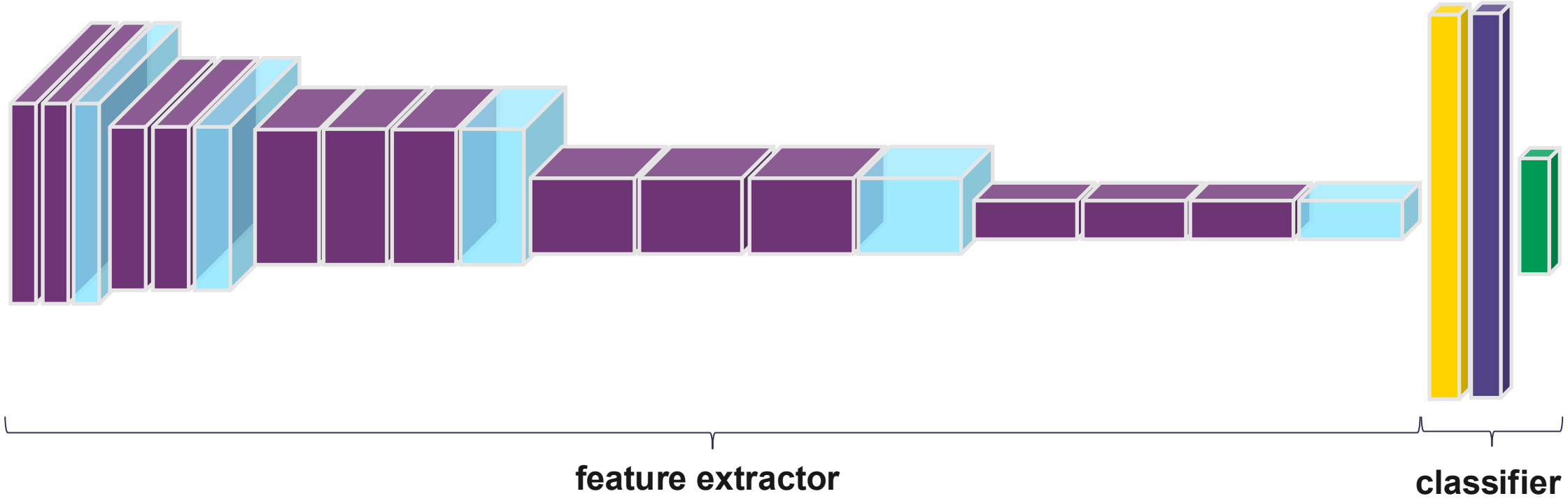
04 May 2026

ASIAN INSTITUTE OF MANAGEMENT

Transfer Learning Approaches

Transfer learning approaches

Pretrained model example - VGG-16



https://arxiv.org/pdf/1409.1556

Conv2D layer
Max pooling layer
Flatten layer
Dense layer
Output layer

Transfer learning approaches

Simple CNN Example

```
initial = models.Sequential([
    layers.Input(shape=(180, 180, 3)),
    layers.Conv2D(32, (3, 3),
                        activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3),
                        activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dense(64,
                 activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(1,
                 activation='sigmoid')
])
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 178, 178, 32)	896
max_pooling2d (MaxPooling2D)	(None,)	
conv2d_1 (Conv2D)	(None,)	
max_pooling2d_1 (MaxPooling2D)	(None,)	
flatten (Flatten)	(None,)	
dense (Dense)	(None, 64)	
dropout (Dropout)	(None, 64)	
dense_1 (Dense)	(None, 1)	

Total params:

Trainable params:

Non-trainable params: 0 (0.00 B)

NUMBER OF PARAMETERS

$$n_{filters} \times filter\ size \times n_{channels} + \text{bias}$$

CONV2D OUTPUT SIZE

$$floor\left(\frac{h_{input} + 2 \cdot h_{filter} - filter\ size}{stride}\right) + 1$$

FLATTENED OUTPUT SIZE

$$h_{input} \times h_{output} \times n_{channels}$$

POOLED OUTPUT SIZE

$$floor\left(\frac{h_{input} - h_{pool\ size}}{stride}\right) + 1$$

Transfer learning approaches

Simple CNN Example

```
initial = models.Sequential([
    layers.Input(shape=(180, 180, 3)),
    layers.Conv2D(32, (3, 3),
                  activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3),
                  activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dense(64,
                 activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(1,
                 activation='sigmoid')
])
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 178, 178, 32)	896
max_pooling2d (MaxPooling2D)	(None, 89, 89, 32)	0
conv2d_1 (Conv2D)	(None, 87, 87, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 43, 43, 64)	0
flatten (Flatten)	(None, 118336)	0
dense (Dense)	(None, 64)	7,573,568
dropout (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 1)	65

Total params: 7,593,025 (28.97 MB)

Trainable params: 7,593,025 (28.97 MB)

Non-trainable params: 0 (0.00 B)

NUMBER OF PARAMETERS

$$n_{filters} \times filter\ size \times n_{channels} + \text{bias}$$

CONV2D OUTPUT SIZE

$$floor\left(\frac{h_{input} + 2 \cdot h_{filter} - filter\ size}{stride}\right) + 1$$

FLATTENED OUTPUT SIZE

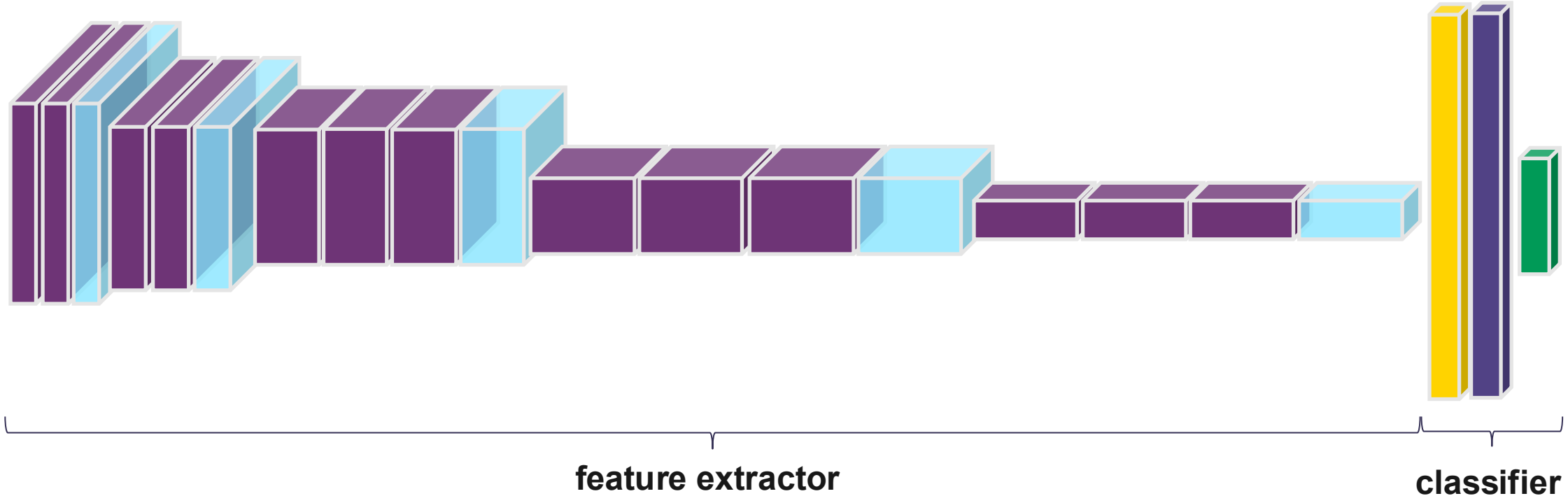
$$h_{input} \times h_{output} \times n_{channels}$$

POOLED OUTPUT SIZE

$$floor\left(\frac{h_{input} - h_{pool\ size}}{stride}\right) + 1$$

Transfer learning approaches

Pretrained model example - VGG-16

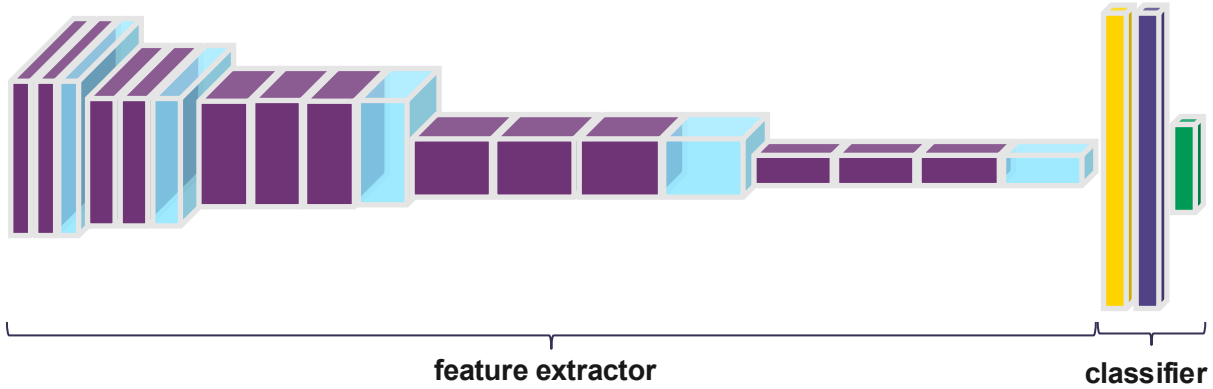


https://arxiv.org/pdf/1409.1556

Conv2D layer
Max pooling layer
Flatten layer
Dense layer
Output layer

Transfer learning approaches

Pretrained model example - VGG-16



Conv2D layer
Max pooling layer
Flatten layer
Dense layer
Output layer

https://arxiv.org/pdf/1409.1556

Layer (type)	Output Shape	Param #
input_layer_2 (InputLayer)	(None, 224, 224, 3)	0
block1_conv1 (Conv2D)	(None, 224, 224, 64)	1,792
block1_conv2 (Conv2D)	(None, 224, 224, 64)	36,928
block1_pool (MaxPooling2D)	(None, 112, 112, 64)	0
block2_conv1 (Conv2D)	(None, 112, 112, 128)	73,856
block2_conv2 (Conv2D)	(None, 112, 112, 128)	147,584
block2_pool (MaxPooling2D)	(None, 56, 56, 128)	0
block3_conv1 (Conv2D)	(None, 56, 56, 256)	295,168
block3_conv2 (Conv2D)	(None, 56, 56, 256)	590,080
block3_conv3 (Conv2D)	(None, 56, 56, 256)	590,080
block3_pool (MaxPooling2D)	(None, 28, 28, 256)	0
block4_conv1 (Conv2D)	(None, 28, 28, 512)	1,180,160
block4_conv2 (Conv2D)	(None, 28, 28, 512)	2,359,808
block4_conv3 (Conv2D)	(None, 28, 28, 512)	2,359,808
block4_pool (MaxPooling2D)	(None, 14, 14, 512)	0
block5_conv1 (Conv2D)	(None, 14, 14, 512)	2,359,808
block5_conv2 (Conv2D)	(None, 14, 14, 512)	2,359,808
block5_conv3 (Conv2D)	(None, 14, 14, 512)	2,359,808
block5_pool (MaxPooling2D)	(None, 7, 7, 512)	0
flatten (Flatten)	(None, 25088)	0
fc1 (Dense)	(None, 4096)	102,764,544
fc2 (Dense)	(None, 4096)	16,781,312
predictions (Dense)	(None, 1000)	4,097,000

Total params: 138,357,544 (527.79 MB)

Trainable params: 138,357,544 (527.79 MB)

Non-trainable params: 0 (0.00 B)

Transfer learning approaches

Pretrained model as a feature extractor

Using the representations learned by a previously trained model to extract interesting features from new samples which are then run through a new classifier

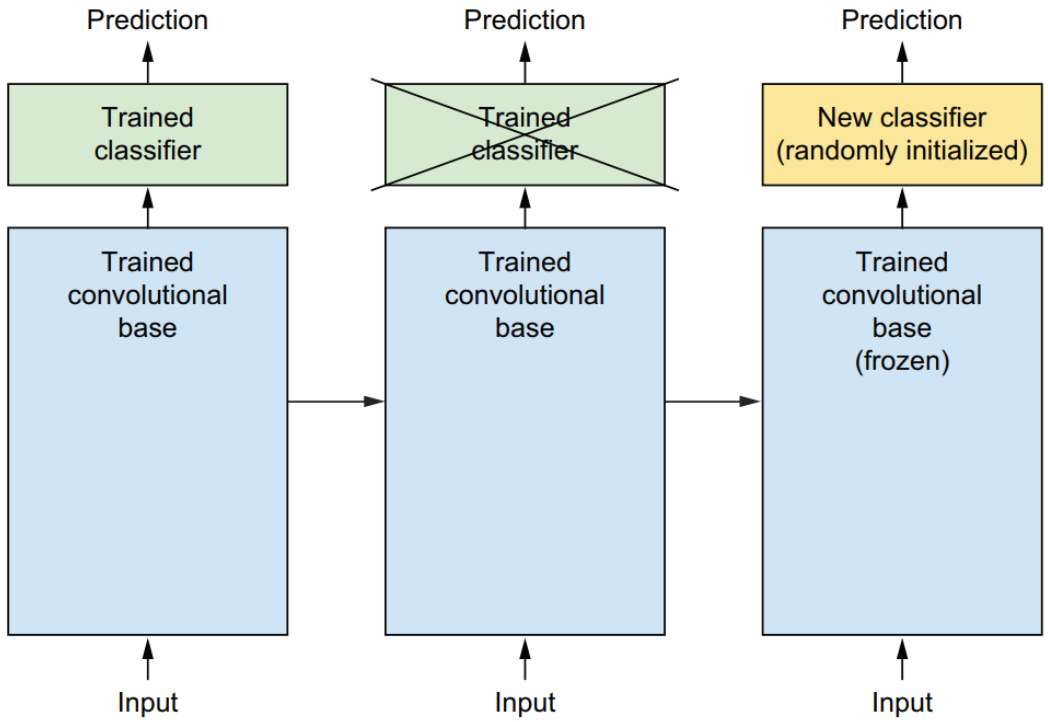
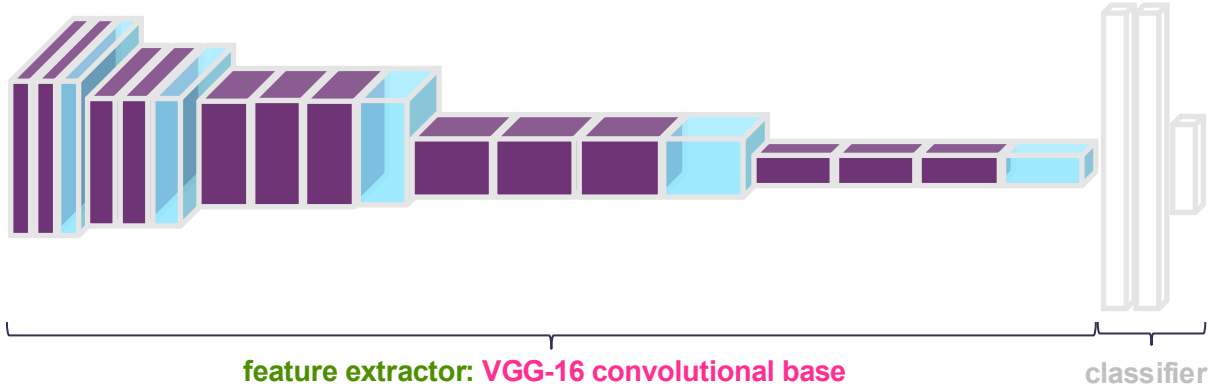


Figure 8.12 Swapping classifiers while keeping the same convolutional base

Chollet, F., & Watson, M. (2026). *Deep Learning with Python Third Edition.* Manning.

Transfer learning approaches

Pretrained model as a feature extractor



Conv2D layer
Max pooling layer
Flatten layer
Dense layer
Output layer

https://arxiv.org/pdf/1409.1556

Layer (type)	Output Shape	Param #
input_layer_2 (InputLayer)	(None, 224, 224, 3)	0
block1_conv1 (Conv2D)	(None, 224, 224, 64)	1,792
block1_conv2 (Conv2D)	(None, 224, 224, 64)	36,928
block1_pool (MaxPooling2D)	(None, 112, 112, 64)	0
block2_conv1 (Conv2D)	(None, 112, 112, 128)	73,856
block2_conv2 (Conv2D)	(None, 112, 112, 128)	147,584
block2_pool (MaxPooling2D)	(None, 56, 56, 128)	0
block3_conv1 (Conv2D)	(None, 56, 56, 256)	295,168
block3_conv2 (Conv2D)	(None, 56, 56, 256)	590,080
block3_conv3 (Conv2D)	(None, 56, 56, 256)	590,080
block3_pool (MaxPooling2D)	(None, 28, 28, 256)	0
block4_conv1 (Conv2D)	(None, 28, 28, 512)	1,180,160
block4_conv2 (Conv2D)	(None, 28, 28, 512)	2,359,808
block4_conv3 (Conv2D)	(None, 28, 28, 512)	2,359,808
block4_pool (MaxPooling2D)	(None, 14, 14, 512)	0
block5_conv1 (Conv2D)	(None, 14, 14, 512)	2,359,808
block5_conv2 (Conv2D)	(None, 14, 14, 512)	2,359,808
block5_conv3 (Conv2D)	(None, 14, 14, 512)	2,359,808
block5_pool (MaxPooling2D)	(None, 7, 7, 512)	0

Total params: 14,714,688 (56.13 MB)

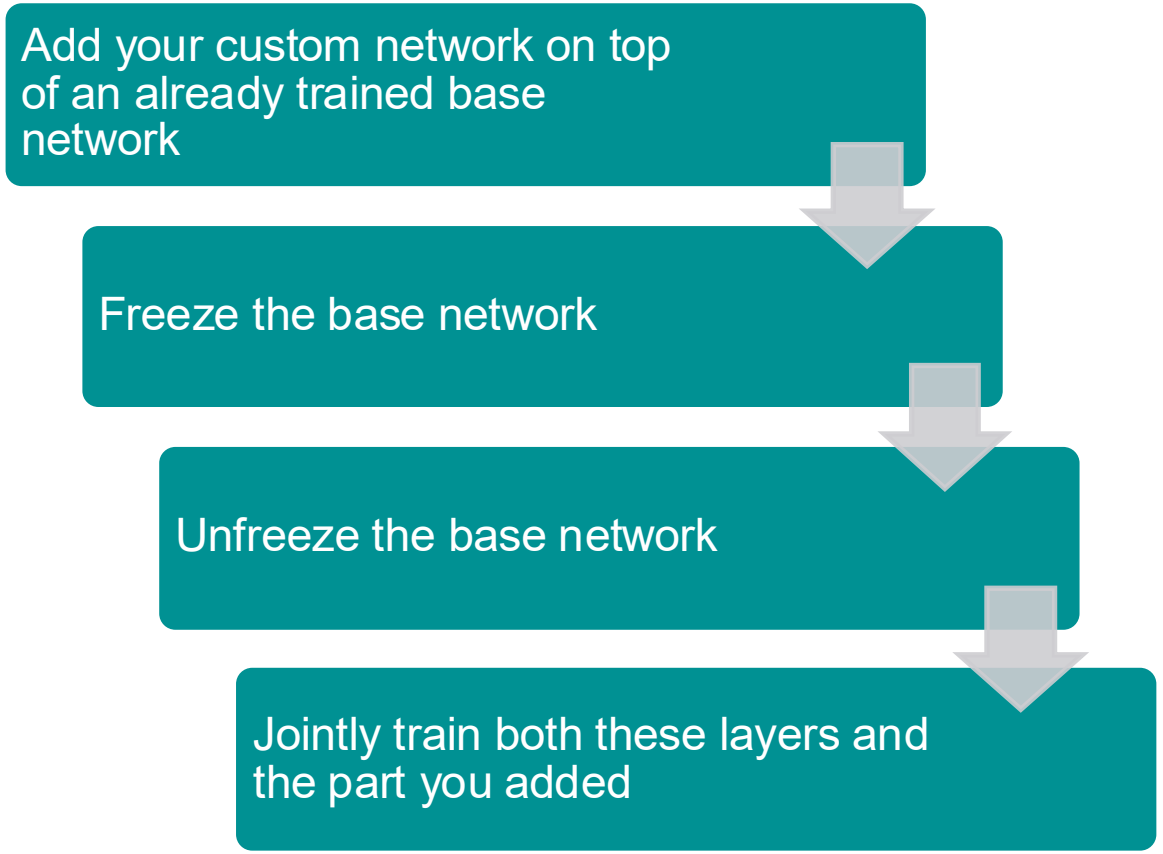
Trainable params: 0 (0.00 B)

Non-trainable params: 14,714,688 (56.13 MB)

Transfer learning approaches

Pretrained model with fine-tuning

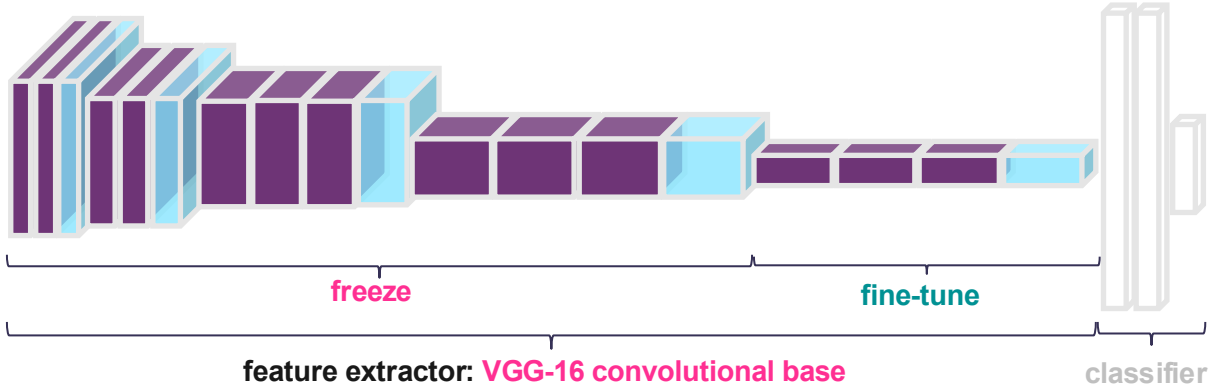
Unfreezing the frozen model base used for feature extraction and jointly training both the newly added part of the model and the base model



Chollet, F., & Watson, M. (2026). *Deep Learning with Python Third Edition.* Manning.

Transfer learning approaches

Pretrained model with fine-tuning



Conv2D layer
Max pooling layer
Flatten layer
Dense layer
Output layer

https://arxiv.org/pdf/1409.1556

Layer (type)	Output Shape	Param #
input_layer_2 (InputLayer)	(None, 224, 224, 3)	0
block1_conv1 (Conv2D)	(None, 224, 224, 64)	1,792
block1_conv2 (Conv2D)	(None, 224, 224, 64)	36,928
block1_pool (MaxPooling2D)	(None, 112, 112, 64)	0
block2_conv1 (Conv2D)	(None, 112, 112, 128)	73,856
block2_conv2 (Conv2D)	(None, 112, 112, 128)	147,584
block2_pool (MaxPooling2D)	(None, 56, 56, 128)	0
block3_conv1 (Conv2D)	(None, 56, 56, 256)	295,168
block3_conv2 (Conv2D)	(None, 56, 56, 256)	590,080
block3_conv3 (Conv2D)	(None, 56, 56, 256)	590,080
block3_pool (MaxPooling2D)	(None, 28, 28, 256)	0
block4_conv1 (Conv2D)	(None, 28, 28, 512)	1,180,160
block4_conv2 (Conv2D)	(None, 28, 28, 512)	2,359,808
block4_conv3 (Conv2D)	(None, 28, 28, 512)	2,359,808
block4_pool (MaxPooling2D)	(None, 14, 14, 512)	0
block5_conv1 (Conv2D)	(None, 14, 14, 512)	2,359,808
block5_conv2 (Conv2D)	(None, 14, 14, 512)	2,359,808
block5_conv3 (Conv2D)	(None, 14, 14, 512)	2,359,808
block5_pool (MaxPooling2D)	(None, 7, 7, 512)	0

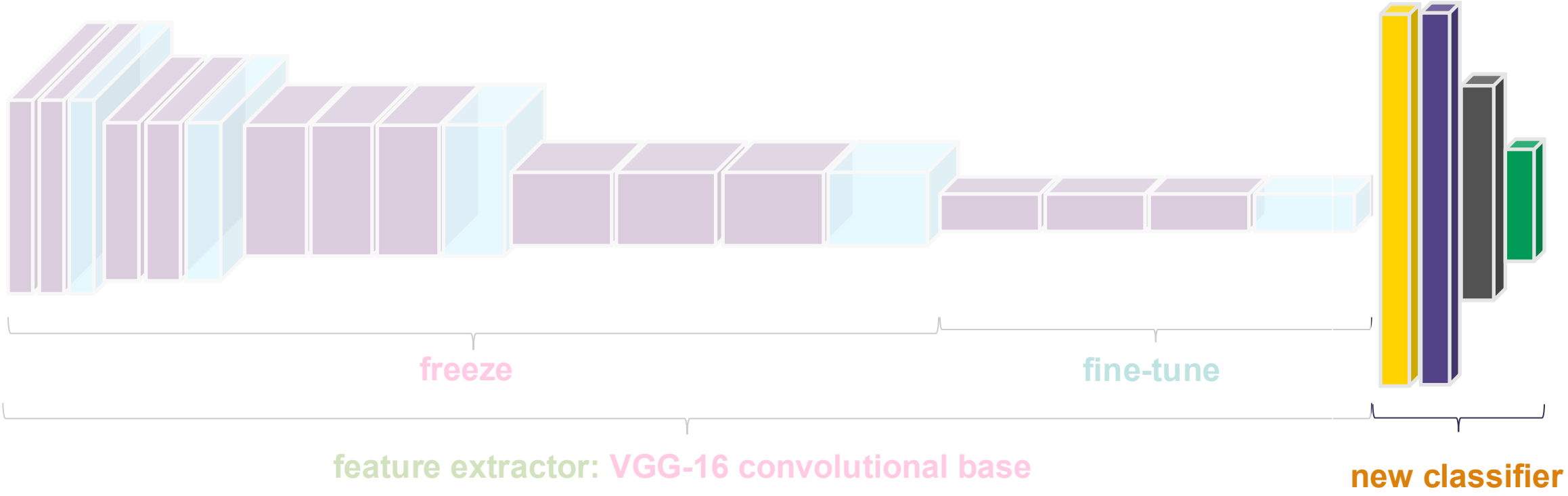
Total params: 14,714,688 (56.13 MB)

Trainable params: 7,079,424 (27.01 MB)

Non-trainable params: 7,635,264 (29.13 MB)

Transfer learning approaches

Pretrained model with a custom network

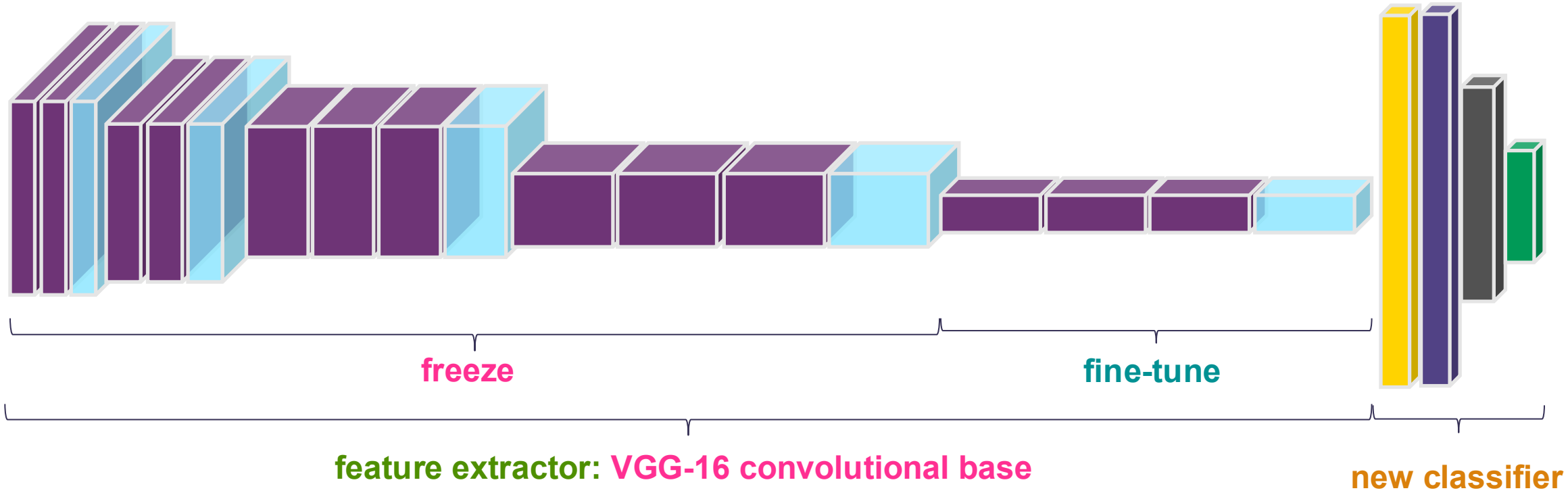


https://arxiv.org/pdf/1409.1556

Conv2D layer
Max pooling layer
Dropout layer
Flatten layer
Dense layer
Output layer

Transfer learning approaches

Pretrained model with a custom network



https://arxiv.org/pdf/1409.1556

Conv2D layer
Max pooling layer
Dropout layer
Flatten layer
Dense layer
Output layer

Transfer learning approaches

Summary

Pretrained Model as a Feature Extractor		Pretrained Model with Fine-Tuning
fast	slow	
▪ Freeze all layers ▪ Extract and store features of the due data first before model training ▪ Design a new set of fully connected layers (classifier)	▪ Freeze all layers ▪ Extract features of the new data during model training ▪ Add a new set of fully connected layers (classifier)	▪ Freeze all layers ▪ Unfreeze layers that would be trained on the new data ▪ Add a new custom network with a set of fully connected layers (classifier)

Architectural Innovations in CNN Pretrained Models

Architectural innovations

AlexNet (2012)

- Deep CNN (8 layers)
- Rectified Linear Units (ReLU)
- Response normalization
- Use of GPU

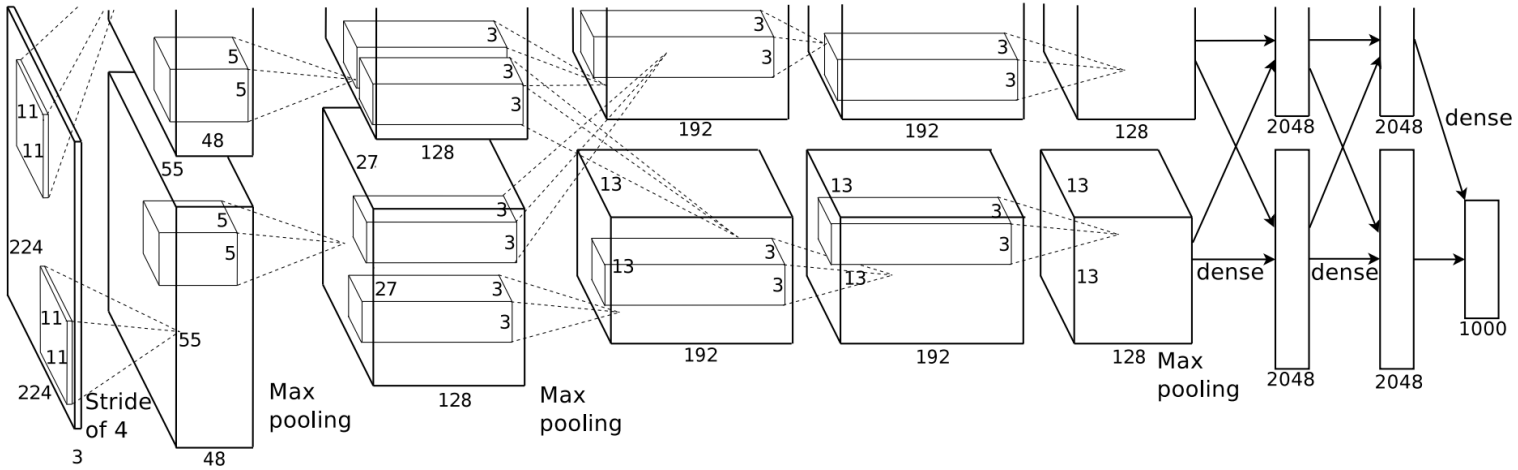


Figure 2: An illustration of the architecture of our CNN, explicitly showing the delineation of responsibilities between the two GPUs. One GPU runs the layer-parts at the top of the figure while the other runs the layer-parts at the bottom. The GPUs communicate only at certain layers. The network's input is 150,528-dimensional, and the number of neurons in the network's remaining layers is given by 253,440–186,624–64,896–64,896–43,264–4096–4096–1000.

https://proceedings.neurips.cc/paper_files/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf

Architectural innovations

VGG (2015)

- Deeper is better
- Keeps it simple by using 3x3 convolutions

Table 1: **ConvNet configurations** (shown in columns). The depth of the configurations increases from the left (A) to the right (E), as more layers are added (the added layers are shown in bold). The convolutional layer parameters are denoted as "conv⟨receptive field size⟩-⟨number of channels⟩". The ReLU activation function is not shown for brevity.

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 **LRN**	conv3-64 **conv3-64**	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 **conv3-128**	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 **conv1-256**	conv3-256 conv3-256 **conv3-256**	conv3-256 conv3-256 conv3-256 **conv3-256**
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 **conv1-512**	conv3-512 conv3-512 **conv3-512**	conv3-512 conv3-512 conv3-512 **conv3-512**
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 **conv1-512**	conv3-512 conv3-512 **conv3-512**	conv3-512 conv3-512 conv3-512 **conv3-512**
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

https://arxiv.org/pdf/1409.1556

Architectural innovations

ResNet (2015)

- Deeper is better, but too deep and the gradients will vanish
- Add residual or skip connections to help with gradient flow
  - **RESIDUAL CONNECTIONS**

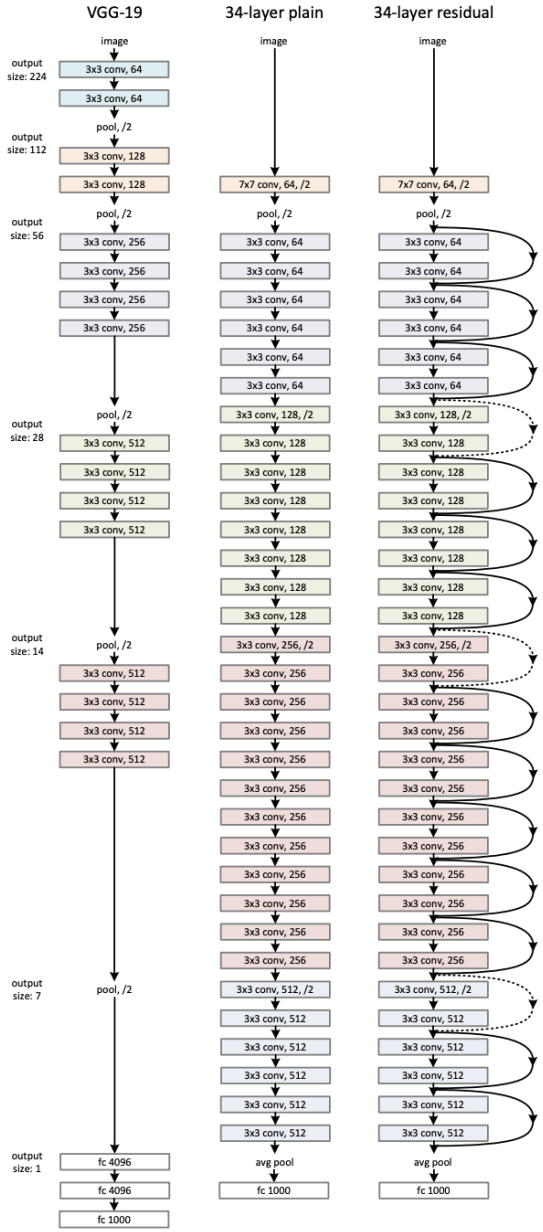
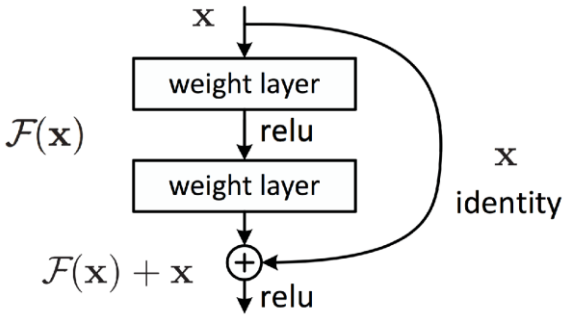


Figure 3. Example network architectures for ImageNet. **Left**: the VGG-19 model [41] (19.6 billion FLOPs) as a reference. **Middle**: a plain network with 34 parameter layers (3.6 billion FLOPs). **Right**: a residual network with 34 parameter layers (3.6 billion FLOPs). The dotted shortcuts increase dimensions. **Table 1** shows more details and other variants.



https://arxiv.org/abs/1512.03385

Architectural innovations

DenseNet (2016)

- More residual connections
- Connects each block to every other block in a feed-forward fashion

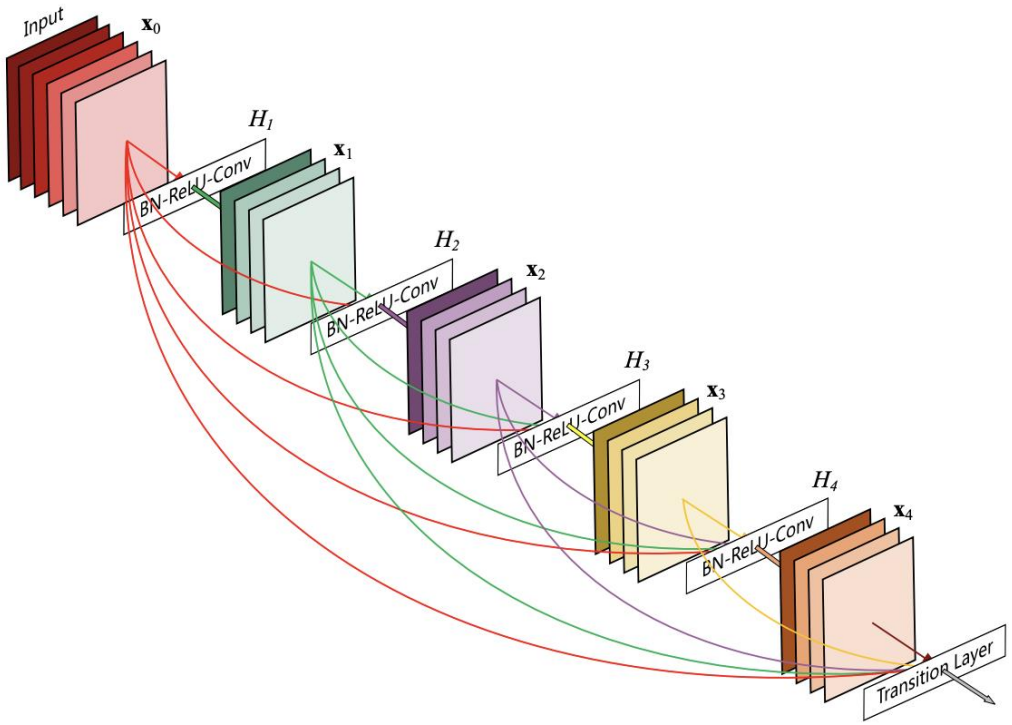


Figure 1: A 5-layer dense block with a growth rate of $k = 4$. Each layer takes all preceding feature-maps as input.

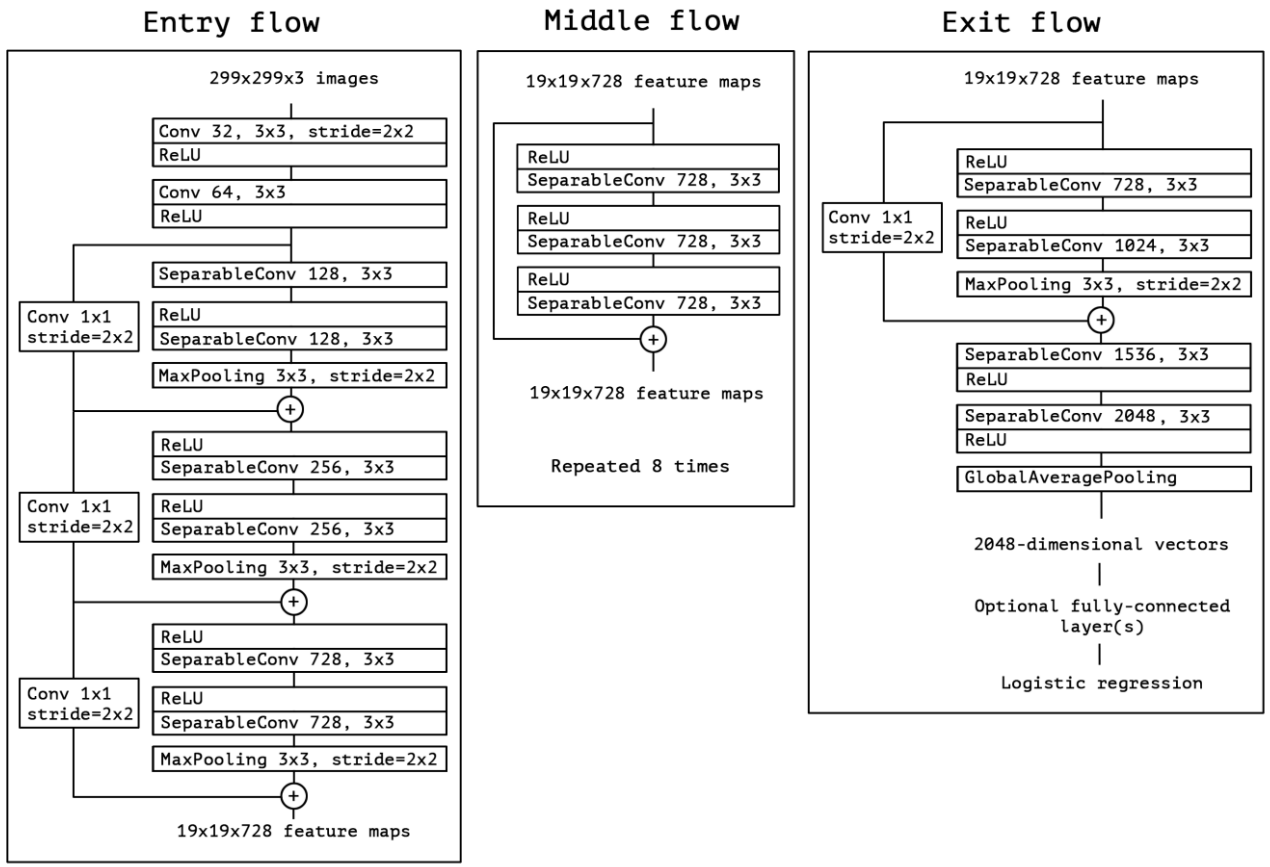
https://arxiv.org/pdf/1608.06993

Architectural innovations

Xception (2017)

- an intermediate step in-between regular convolution and the depth-wise separable convolution operation
  - **DEPTH-WISE SEPARABLE CONVOLUTION**

Figure 5. The Xception architecture: the data first goes through the entry flow, then through the middle flow which is repeated eight times, and finally through the exit flow. Note that all Convolution and SeparableConvolution layers are followed by batch normalization [7] (not included in the diagram). All SeparableConvolution layers use a depth multiplier of 1 (no depth expansion).



https://arxiv.org/pdf/1610.02357

Architectural innovations

EfficientNet (2019)

- carefully balancing network depth, width, and resolution can leac to better performance
  - **COMPOUND SCALING**

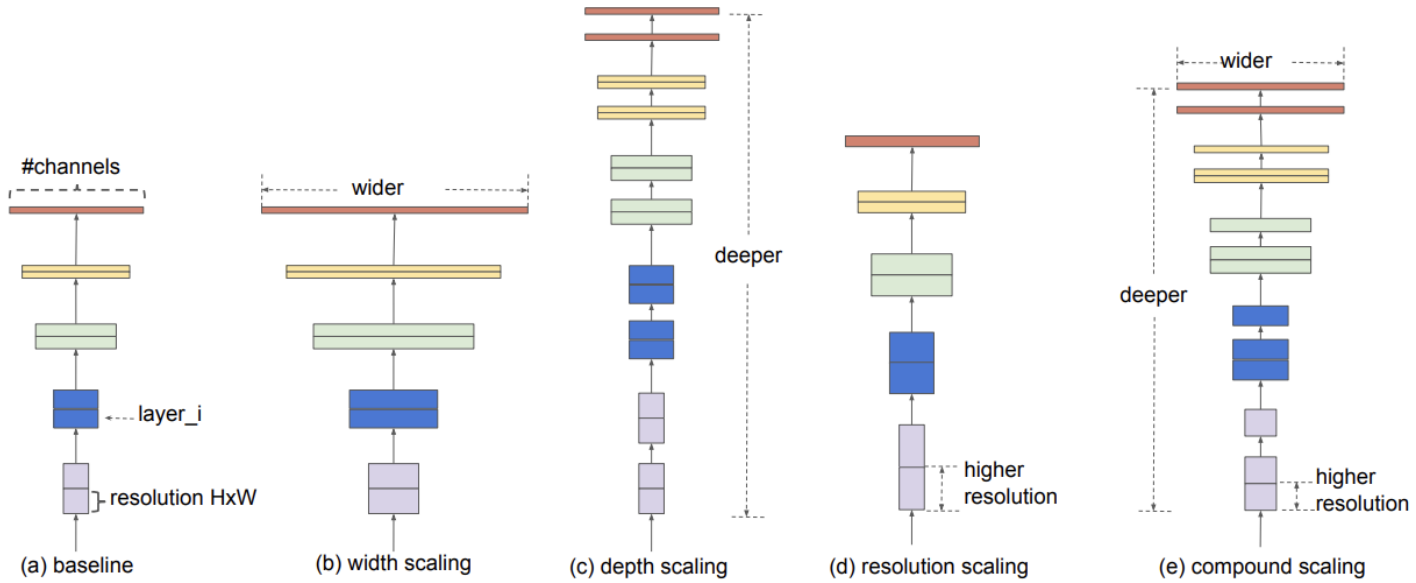


Figure 2. **Model Scaling.** (a) is a baseline network example; (b)-(d) are conventional scaling that only increases one dimension of network width, depth, or resolution. (e) is our proposed compound scaling method that uniformly scales all three dimensions with a fixed ratio.

https://arxiv.org/pdf/1905.11946

Exercise 5:
In-class Competition

Regularization

Exercise 4

INSTRUCTIONS

1. Create a CNN that classifies 🍽 food images containing 10 classes. The dataset that we will use is a subset of the Food-101 dataset from Kaggle, which contains labeled food images.

2. The data preparation and creation of a baseline CNN is already provided. Improve the performance of the baseline model by applying the regularization techniques discussed.

LT X (feel free to change the slide visuals)

Final Performance (accuracy)

Paste performance trends per epoch

Architecture

- Pretrained model
- Classifier
- Number of parameters (paste model summary)

Learnings from the hackathon

Reminders

Reminders

Next Session

Session 9 and 10 (May 8, 2026) –RNN

Deliverables

Due this week

- R04 - Post-Class Reflection 4 (due EOD May 4, 2026)
- E5 - Exercise 5 (due by the end of the class – May 4, 2026)

Due two weeks from now

- M1 - Mini-project 1 (due EOD May 18, 2026)

ASIAN INSTITUTE OF MANAGEMENT

Machine Learning 3

Convolutional Neural Networks

Sessions 7-8

MSDS 2026

04 May 2026

ASIAN INSTITUTE OF MANAGEMENT